

LAB-4: Priklausomybiu Izoliavimas (Visi Variantai)

Tikslas: Isplesti LAB-3 projekta taip, kad pasirinkta strategija butu izoliuota nuo konkrečiu implementaciju ir leistu kontroliuoti priklausomybiu elgesi naudojant Fake ir Stub.

Bendri reikalavimai:

- Program.cs dirba tik su interface
- Jokios verslo logikos Main metode
- Jokio new klases viduje (naudoti constructor injection)
- Sukurtos Fake ir Stub implementacijos
- Pademonstruotos bent 2 logikos sakos
- README paaiskina kas izoliuojama ir kodel

VARIANTAS 1 – Studentu rodymo strategija (IStudentPrinter)

Situacija: Spausdinimo mechanizmas ateityje gali buti pakeistas (pvz. is Console i Faila ar API). Reikia izoliuoti rodymo logika nuo verslo logikos.

```
public interface IStudentPrinter
{
    void Print(Group group);
}
```

Reikalavimai: 1. Izoliuoti sia priklausomybe per konstruktoriu. 2. Sukurti Fake klase (stabilus rezultatas). 3. Sukurti Stub klase (kontroliuojamas rezultatas). 4. Pademonstruoti bent 2 verslo logikos scenarijus. 5. Paaiskinti README faile kokia problema buvo sprendziama.

VARIANTAS 2 – Vidurkio skaiciavimo strategija (IAverageStrategy)

Situacija: Vidurkio skaiciavimo taisykles gali keistis (Simple, Weighted ir pan.). Reikia izoliuoti skaiciavimo mechanizma nuo StudentService.

```
public interface IAverageStrategy
{
    double Calculate(Student student);
}
```

Reikalavimai: 1. Izoliuoti sia priklausomybe per konstruktoriu. 2. Sukurti Fake klase (stabilus rezultatas). 3. Sukurti Stub klase (kontroliuojamas rezultatas). 4. Pademonstruoti bent 2 verslo logikos scenarijus. 5. Paaiskinti README faile kokia problema buvo sprendziama.

VARIANTAS 3 – Meniu abstrakcija (IMenuService)

Situacija: Meniu mechanizmas gali buti pakeistas (Basic, Advanced, Web UI ir pan.). Reikia izoliuoti vartotojo sąsajos vykdymą nuo programos branduolio.

```
public interface IMenuService
{
    void Run();
}
```

Reikalavimai: 1. Izoliuoti šią priklausomybę per konstruktoriu. 2. Sukurti Fake klasę (stabilus rezultatas). 3. Sukurti Stub klasę (kontroliuojamas rezultatas). 4. Pademonstruoti bent 2 verslo logikos scenarijus. 5. Paaiskinti README faile kokia problema buvo sprendžiama.

VARIANTAS 4 – Paieskos strategija (IStudentFinder)

Situacija: Paieska gali buti vykdoma pagal ID, Email ar isorine API. Reikia izoliuoti paieskos mechanizma nuo verslo logikos.

```
public interface IStudentFinder
{
    Student? Find(Group group, string query);
}
```

Reikalavimai: 1. Izoliuoti sia priklausomybe per konstruktoriu. 2. Sukurti Fake klase (stabilus rezultatas). 3. Sukurti Stub klase (kontroliuojamas rezultatas). 4. Pademonstruoti bent 2 verslo logikos scenarijus. 5. Paaiskinti README faile kokia problema buvo sprendziama.

VARIANTAS 5 – Validacija (IStudentValidator)

Situacija: Validavimo taisyklės gali būti griežtos arba lankstesnės. Reikia izoliuoti validavimo taisyklės nuo StudentService.

```
public interface IStudentValidator
{
    bool Validate(Student student);
}
```

Reikalavimai: 1. Izoliuoti šią priklausomybę per konstruktorių. 2. Sukurti Fake klasę (stabilus rezultatas). 3. Sukurti Stub klasę (kontroliuojamas rezultatas). 4. Pademonstruoti bent 2 verslo logikos scenarijus. 5. Paaiskinti README faile kokia problema buvo sprendžiama.

Vertinimas (10 balu):

2 balai - Teisingas priklausomybes izoliavimas 2 balai - Fake implementacija 2 balai - Stub implementacija 2 balai - Aiskiai pademonstruotos logikos sakos 2 balai - README su architekturiniu paaiskinimu